Imports

In [448…
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import base64
```

In [ ]:
```
```

In [449…
```
import ratinabox
from ratinabox.Environment import Environment
from ratinabox.Agent import Agent
from ratinabox.Neurons import FieldOfViewOVCs
# from ratinabox.Neurons import PlaceCells, GridCells #...
```

aux funcs

In [450…
```python
import random
import string

def generate_random_string(n):
    """
    Generate a random string of length n.

    Parameters:
    n (int): The desired length of the string.

    Returns:
    str: A random string of length n.
    """
    if n <= 0:
        raise ValueError("Length 'n' must be a positive integer.")

    # Combine letters, digits, and punctuation
    characters = string.ascii_uppercase

    # Generate a random string of length n
    random_string = ''.join(random.choices(characters, k=n))

    return random_string
```

In [451…
```python
generate_random_string(8)
```

Out[451… 'KAXNSVQF'

In [452…
```python
def get_rand_letter(x):
    if x <= 0.2:
        return 'A'
    elif x > 0.2 and x <= 0.4:
        return 'B'
    elif x > 0.4 and x <= 0.6:
        return 'C'
    elif x > 0.6 and x <= 0.8:
        return 'D'
    else:
        return 'E'
```

In [453…
```python
def is_point_in_hole(point, hole):
    """
    Verifica se um ponto está dentro de um polígono (hole).
    """
    x, y = point
```

```
    n = len(hole)
    inside = False

    px, py = hole[0]
    for i in range(1, n + 1):
        cx, cy = hole[i % n]
        if y > min(py, cy):
            if y <= max(py, cy):
                if x <= max(px, cx):
                    if py != cy:
                        xinters = (y - py) * (cx - px) / (cy - py) + px
                        if px == cx or x <= xinters:
                            inside = not inside
        px, py = cx, cy

    return inside
```

In [454…

```
def is_point_valid(point, hole):
    x_valid = True
    y_valid = True
    point_valid = True
    hole_x_inf = hole[0][0]
    hole_x_sup = hole[1][0]
    hole_y_inf = hole[0][1]
    hole_y_sup = hole[1][1]
    # check x pos
    if (point[0] > hole_x_inf) and (point[0] < hole_x_sup):
        x_valid = False
    if (point[1] > hole_y_inf) and (point[1] < hole_y_sup):
        y_valid = False
    if (not x_valid or  not y_valid) or (not x_valid and y_valid):
        point_valid = False
    # print(point, point_valid, x_valid, y_valid)
    return point_valid
```

In [455…

```
def get_euclidean_distance(pos1, pos2):
    return np.sqrt((pos1[0] - pos2[0])**2 + ((pos1[1] - pos2[1])**2))
def apply_eucd_dist(x, obj=9):
    return get_euclidean_distance(x['pos'], x[f'obj_{obj}_pos'])
```

In [456…

```
hole_deff = [[0.4,0.2],[0.6,0.2],[0.6,0.8],[0.4,0.8]]
```

In [457…

```
hole_deff[1][0]
```

Out[457… 0.6

Define EnV

In [458…
```
Env = Environment(params={
    'aspect':1,
    'holes' : [[[0.4,0.2],[0.6,0.2],[0.6,0.8],[0.4,0.8]]],
    })
```

Num of Cues

In [459…
```
NUM_OF_CUES = 100
```

gerando cues positions

In [460…
```
CUES_LIST = []
num_test = 0
for i in range(NUM_OF_CUES):
    candidate_point = 0.45, 0.55
    while not is_point_valid(candidate_point, hole_deff):
        candidate_point = np.random.rand(), np.random.rand()
        num_test+=1
    CUES_LIST.append(candidate_point)
print(f'{NUM_OF_CUES} cues criadas com {num_test} candidatos')
```

```
100 cues criadas com 130 candidatos
```

adicionando cues ao EnV

In [461…
```
for point in CUES_LIST:
    Env.add_object(object=point,type=0)
```

In [462…
```
len(Env.objects['objects']), NUM_OF_CUES
```

Out[462… (100, 100)

adding name to cue

cues name len

In [463…
```
CUE_NAME_LEN = 8
```

In [464…
```
objects_name_list = []
for i in range(NUM_OF_CUES):
    obj_name = generate_random_string(CUE_NAME_LEN)
    if obj_name in objects_name_list:
        obj_name = generate_random_string(CUE_NAME_LEN)
    objects_name_list.append(obj_name)
```

In [465…
```
len(objects_name_list), len(set(objects_name_list))
```

Out[465… (100, 100)

In [466…
```
# # objetos a direita do buraco
# Env.add_object(object=[0.9,0.3],type=0)
# Env.add_object(object=[0.7,0.1],type=0)
# Env.add_object(object=[0.7,0.5],type=0)
# Env.add_object(object=[0.9,0.7],type=0)
# Env.add_object(object=[0.7,0.9],type=0)


# # objetos a esquerda do buraco
# Env.add_object(object=[0.3,0.3],type=0)
# Env.add_object(object=[0.1,0.1],type=0)
# Env.add_object(object=[0.1,0.5],type=0)
# Env.add_object(object=[0.3,0.7],type=0)
# Env.add_object(object=[0.1,0.9],type=0)
```

In [467…
```
objects_name_list[:3]
```

Out[467… ['OLARNHNK', 'TGMGEDNO', 'HOYDPVZZ']

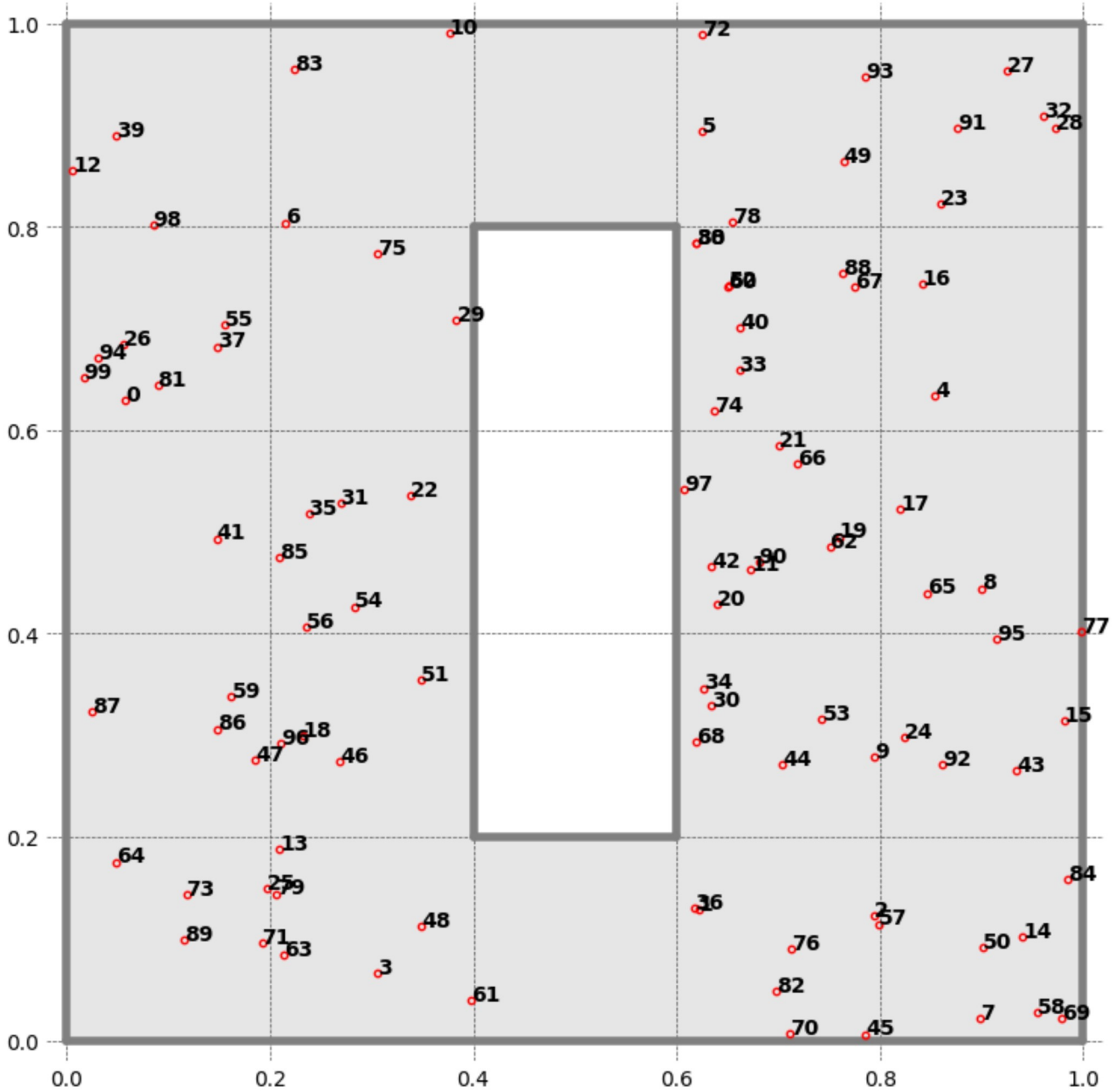
In [468…
```
Env.objects['object_names'] = objects_name_list
```

In [469…
```
# Env.objects
```

vizz env

In [470…
```
fig, ax = plt.subplots(1, 1, figsize=(14,7))
```

```
for i in range(NUM_OF_CUES):
    plt.annotate(text=i, xy=Env.objects['objects'][i],fontweight='bold', fontsize=10)
Env.plot_environment(gridlines=True, fig=fig, ax=ax) #visualises current environment with walls and objects
# Agent.plot_trajectory() #plots trajectory
# Agent.animate_trajectory() #animate trajectory
# Neurons.plot_rate_map() # plots the rate map of the neurons at all positions
# Neurons.plot_rate_timeseries() # plots activities of the neurons over time
# Neurons.animate_rate_timeseries() # animates the activity of the neurons over time
plt.show()
```



In [471…
```
Ag = Agent(Env)
```

```
# agent configs
# Ag.pos = start_pos
Ag.thigmotaxis = 0.2
```

In [472…
```
# default_params = {
#         "distance_range": [0.02, 0.4],  # min and max distances the agent can "see"
#         "angle_range": [0,75],  # angluar FoV in degrees (will be symmetric on both sides, so give range in 0 (forwards) to 180 (backwards)
#         "spatial_resolution": 0.02,  # resolution of each BVC tiling FoV
#         "beta": 5, # smaller means larger rate of increase of cell size with radius in hartley type manifolds
#         "cell_arrangement": "diverging_manifold",  # whether all cells have "uniform" receptive field sizes or they grow ("hartley") with radi
#         "object_tuning_type" : None, #can be "random", any integer, or a list of integers of length n. The tuning types of the OVCs.
#     }
```

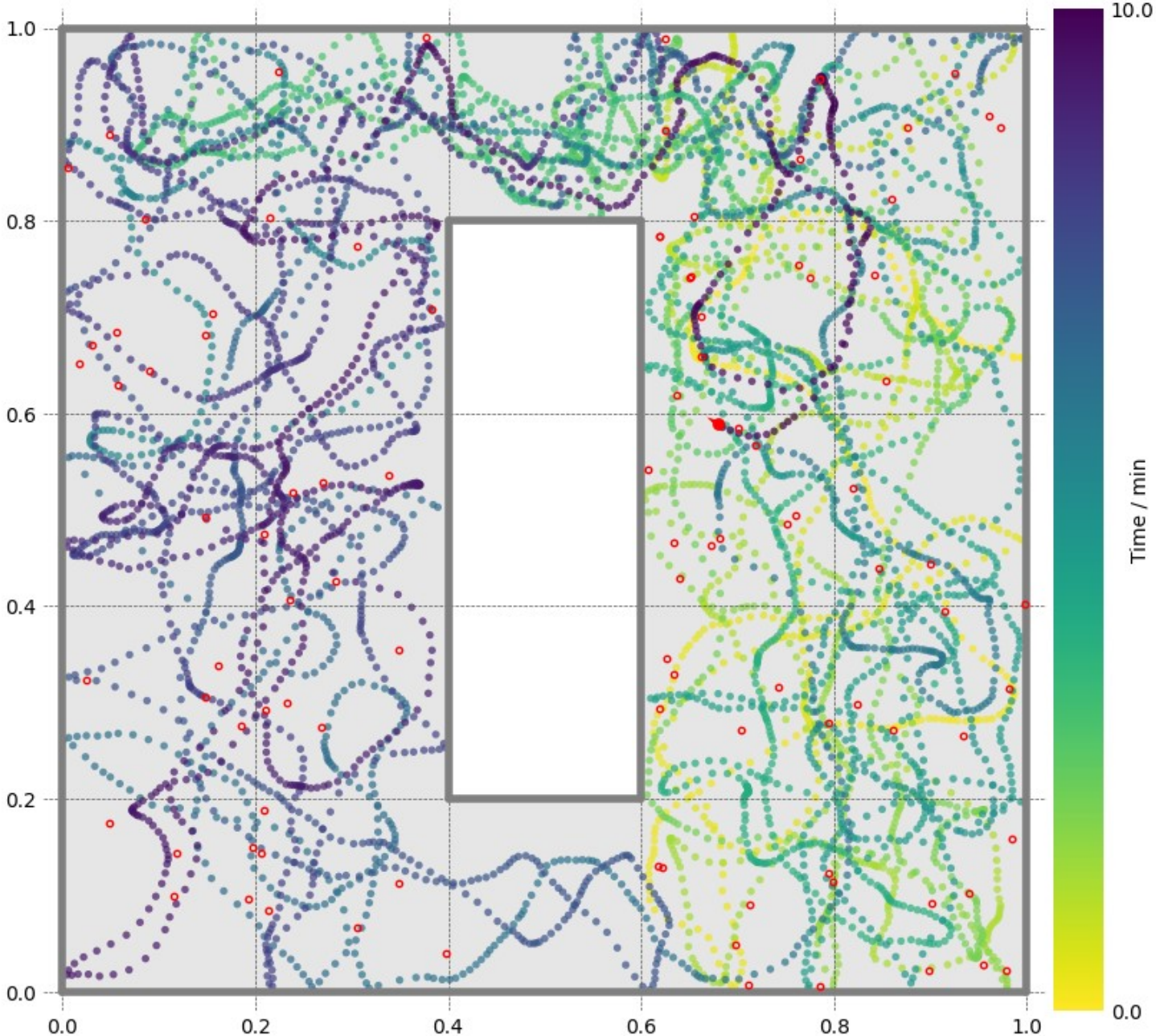
In [ ]:
```
```

In [473…
```
# new_params = {
#         "distance_range": [0.001, 1],  # min and max distances the agent can "see"
#         "angle_range": [0,180],  # angluar FoV in degrees (will be symmetric on both sides, so give range in 0 (forwards) to 180 (backwards)
#         "spatial_resolution": 0.02,  # resolution of each BVC tiling FoV
#         "beta": 5, # smaller means larger rate of increase of cell size with radius in hartley type manifolds
#         "cell_arrangement": "diverging_manifold",  # whether all cells have "uniform" receptive field sizes or they grow ("hartley") with radi
#         "object_tuning_type" : 0, #can be "random", any integer, or a list of integers of length n. The tuning types of the OVCs.
#     }
```

In [474…
```
# FoV_OVCs = FieldOfViewOVCs(Ag,new_params)
```

In [475…
```
for i in range(int(10*60/Ag.dt)):
    Ag.update()
    # FoV_OVCs.update()
```

In [476…
```
fig, ax = plt.subplots(1, 1, figsize=(14,7))
ax = Ag.plot_trajectory(color='changing', colorbar=True, gridlines=True,fig=fig, ax=ax)
plt.show()
```


1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
10.0
Time / min
0.0

In [477…
```
dfbase = pd.DataFrame(Ag.history).copy()
print(dfbase.shape)
dfbase.head(2)
```

```
(12000, 6)
```

Out[477…

	t	pos	distance_travelled	vel	rot_vel	head_direction
0	0.05	[0.9867874608054714, 0.6577358259686839]	0.005453	[0.10620249203552312, 0.02481387637044863]	-2.206043	[0.9544211722924825, 0.29846310639649165]
1	0.10	[0.9913998869974684, 0.6579680388601125]	0.010071	[0.09224852383993953, 0.00464425782857214]	-3.584502	[0.9761102711758364, 0.2172757200080933]

In [478…
```
# df = dfbase.iloc[:150].copy() #dfbase[dfbase.index%10 == 0].copy()
df = dfbase.copy()
df.shape
```

Out[478… (12000, 6)

In [479…
```
# Env.objects
```

In [480…
```
# fixed_object = 8
# Env.objects['objects'][fixed_object]
```

In [481…
```
# Env.objects
```

In [334…

In [482…
```
for i in range(NUM_OF_CUES):
    cue_dict = {f'obj_{i}_pos':[Env.objects['objects'][i] for _ in df.index],
         f'obj_{i}_name':[Env.objects['object_names'][i] for _ in df.index]}
    cue_df = pd.DataFrame(data=cue_dict, index=df.index)
    df = pd.concat([df, cue_df], axis=1)
    # df[f'obj_{i}_pos'] = [Env.objects['objects'][i] for _ in df.index]
    # df[f'obj_{i}_name'] = [Env.objects['object_names'][i] for _ in df.index]
```

In [483…
```
df.shape
```

Out[483… (12000, 206)

In [484…
```
df.head(2)
```

Out[484…

	t	pos	distance_travelled	vel	rot_vel	head_direction	obj_0_pos	obj_0_name	obj_1
0	0.05	[0.9867874608054714, 0.6577358259686839]	0.005453	[0.10620249203552312, 0.02481387637044863]	-2.206043	[0.9544211722924825, 0.29846310639649165]	[0.057799288879790134, 0.6287846804793026]	OLARNHNK	[0.621466145638 0.129140163227
1	0.10	[0.9913998869974684, 0.6579680388601125]	0.010071	[0.09224852383993953, 0.00464425782857214]	-3.584502	[0.9761102711758364, 0.2172757200080933]	[0.057799288879790134, 0.6287846804793026]	OLARNHNK	[0.621466145638 0.129140163227

2 rows × 206 columns

In [485…
```
for i in range(NUM_OF_CUES):
    dist_cue_dict = {f'dist_to_obj_{i}': df.apply(apply_eucd_dist, axis=1, obj=i)}
    dist_cue_df = pd.DataFrame(data=dist_cue_dict, index=df.index)
    df = pd.concat([df, dist_cue_df], axis=1)
```

In [486…
```
f"{100*len(df)} iteracoes"
```

Out[486… '1200000 iteracoes'

In [487…
```
df.shape
```

Out[487… (12000, 306)

In [488…
```
df.tail(2)
```

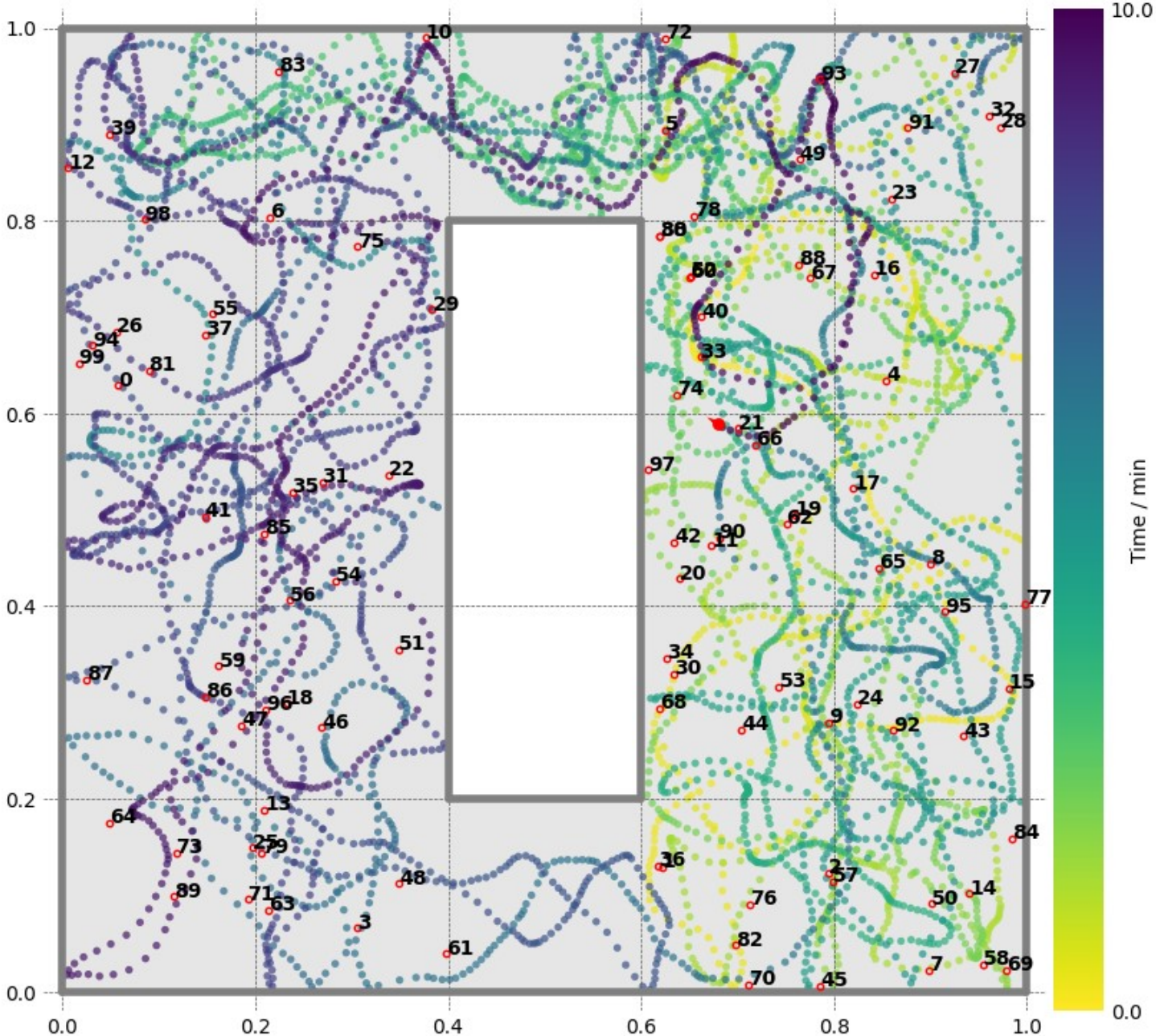
Out[488…

	t	pos	distance_travelled	vel	rot_vel	head_direction	obj_0_pos	obj_0_name	
11998	599.95	[0.6807186768174504, 0.588808729552024]	59.377607	[-0.027537563216450955, 0.03185033037861773]	-4.972701	[-0.8525069854479495, 0.5227158307937972]	[0.057799288879790134, 0.6287846804793026]	OLARNHNK	[0.6214 0.1291
11999	600.00	[0.6788185979727792, 0.5906924394828289]	59.380282	[-0.03800157689342276, 0.037674198616097776]	1.536478	[-0.8098537995726263, 0.5866317612589522]	[0.057799288879790134, 0.6287846804793026]	OLARNHNK	[0.6214 0.1291

2 rows × 306 columns

In [489…
```
fig, ax = plt.subplots(1, 1, figsize=(14,7))
for i in range(NUM_OF_CUES):
    plt.annotate(text=i, xy=Env.objects['objects'][i], fontsize=10, fontweight='bold')
ax = Ag.plot_trajectory(color='changing', colorbar=True, gridlines=True,fig=fig, ax=ax, t_start=0, t_end=df['t'].max())

plt.show()
```


Time / min
10.0
0.0
0.0
0.2
0.4
0.6
0.8
1.0

In [490…
```
obj_names = [Env.objects['object_names'][i] for i in range(NUM_OF_CUES)]
obj_dist_cols = [f'dist_to_obj_{i}' for i in range(NUM_OF_CUES)]
# def get_position_text(x):
```

In [491…
```
dft = df[obj_dist_cols].T
dft.columns = [f't_{i:08d}' for i in range(df.T.shape[1])]
```

In [492…
```
MAX_NUMBER_OF_CUES = 8
CONTEXT_LEN = MAX_NUMBER_OF_CUES*CUE_NAME_LEN
CONTEXT_LEN
```

Out[492… 64

In [493…
```
dft.columns[0]
```

Out[493… 't_00000000'

In [494…
```
order = [int(val.split('_')[-1]) for val in dft.sort_values(dft.columns[0]).index[:MAX_NUMBER_OF_CUES]]
dft.sort_values(dft.columns[0])
```

Out[494…

	t_00000000	t_00000001	t_00000002	t_00000003	t_00000004	t_00000005	t_00000006	t_00000007	t_00000008	t_00000009	...	t_00011990
dist_to_obj_4	0.135225	0.139807	0.142983	0.146051	0.148184	0.146217	0.142366	0.138331	0.134792	0.129844	...	0.153299
dist_to_obj_16	0.169009	0.172876	0.175650	0.178449	0.180667	0.178894	0.175177	0.171201	0.167595	0.161973	...	0.211950
dist_to_obj_23	0.208183	0.210848	0.212837	0.214934	0.216776	0.215455	0.212541	0.209401	0.206511	0.201740	...	0.287275
dist_to_obj_17	0.215199	0.218944	0.221501	0.223906	0.225386	0.223815	0.220952	0.218048	0.215617	0.212672	...	0.121047
dist_to_obj_67	0.227421	0.231642	0.234635	0.237615	0.239901	0.238011	0.234107	0.229941	0.226183	0.220448	...	0.175545
...	...	...	...	...	...	...	...	...	...	...	...	...
dist_to_obj_12	1.000543	1.005020	1.008165	1.011262	1.013565	1.011596	1.007592	1.003337	0.999526	0.993839	...	0.758520
dist_to_obj_73	1.009575	1.013666	1.016460	1.019099	1.020779	1.019073	1.015879	1.012578	1.009740	1.006035	...	0.734677
dist_to_obj_87	1.018487	1.022920	1.025973	1.028894	1.030850	1.028972	1.025363	1.021595	1.018310	1.013808	...	0.731953
dist_to_obj_89	1.035296	1.039306	1.042040	1.044616	1.046238	1.044571	1.041468	1.038266	1.035523	1.031981	...	0.764017
dist_to_obj_64	1.055128	1.059339	1.062221	1.064955	1.066724	1.064960	1.061629	1.058175	1.055191	1.051232	...	0.775048

100 rows × 12000 columns

In [495…
```
order
```

Out[495…
```
[4, 16, 23, 17, 67, 8, 28, 88]
```

In [496…
```
pos_text = ''.join([obj_names[i] for i in order])
pos_text
```

Out[496…
```
'OZIGIIJJRRGIJIWCXAPPPOFNPZGDOBLKLYJJZMWAOXOVBYGXRWGDLJLCLCMNXAUU'
```

In [497…
```
# Env.objects['object_names']
```

In [498…
```
positions_list = []
for feat in dft.columns:
    order = [int(val.split('_')[-1]) for val in dft.sort_values(feat).index[:MAX_NUMBER_OF_CUES]]
    positions_list.append(''.join([obj_names[i] for i in order]))
len(positions_list)
```

Out[498…
```
12000
```

In [499…
```
pos_state_dict = {'position_state':[val for val in positions_list]}
pos_state_df = pd.DataFrame(data=pos_state_dict, index=df.index)
df = pd.concat([df, pos_state_df], axis=1)
df.head(2)
```

Out[499…

	t	pos	distance_travelled	vel	rot_vel	head_direction	obj_0_pos	obj_0_name	obj_1
0	0.05	[0.9867874608054714, 0.6577358259686839]	0.005453	[0.10620249203552312, 0.02481387637044863]	-2.206043	[0.9544211722924825, 0.29846310639649165]	[0.057799288879790134, 0.6287846804793026]	OLARNHNK	[0.621466145638 0.129140163227
1	0.10	[0.9913998869974684, 0.6579680388601125]	0.010071	[0.09224852383993953, 0.00464425782857214]	-3.584502	[0.9761102711758364, 0.2172757200080933]	[0.057799288879790134, 0.6287846804793026]	OLARNHNK	[0.621466145638 0.129140163227

2 rows × 307 columns

In [500…
```
len(df), df['position_state'].nunique()/len(df), df['position_state'].nunique()
```

Out[500…
```
(12000, 0.37316666666666665, 4478)
```

In [ ]:
```

```

In [501…
```
# help(Ag.plot_trajectory)
```

In [502…
```
df['x_pos'] = df['pos'].explode()[::2]
df['y_pos'] = df['pos'].explode()[1::2]
```

```
/tmp/ipykernel_1360/2856337820.py:1: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` m
any times, which has poor performance.  Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use
`newframe = frame.copy()`
  df['x_pos'] = df['pos'].explode()[::2]
/tmp/ipykernel_1360/2856337820.py:2: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` m
any times, which has poor performance.  Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use
`newframe = frame.copy()`
  df['y_pos'] = df['pos'].explode()[1::2]
```

In [503…
```
df['y_pos'].max()
```

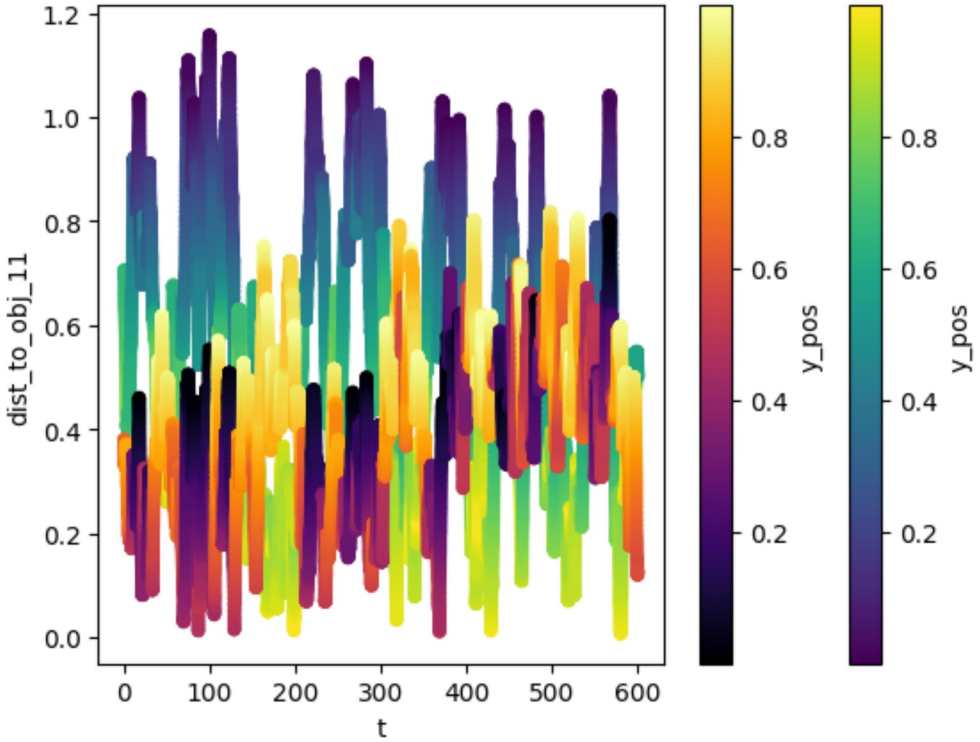
Out[503… 0.9996729956996431

In [504…
```
# df['pos'].explode()[::2]
```

In [505…
```
df.to_csv(r'/home/mgteus/workspace/navitrans/results_100_objs.csv', index=False)
```

In [506…
```
FIXED_OBJECT = 10
```

In [507…
```
fig, ax = plt.subplots()
df.plot.scatter(x='t', y=f'dist_to_obj_{FIXED_OBJECT}', c='y_pos',ax=ax)
df.plot.scatter(x='t', y=f'dist_to_obj_{FIXED_OBJECT+1}', c='y_pos',ax=ax, cmap='inferno')
plt.show()
```



In [508…
```
# df['last_position_state'] = df['position_state'].shift(1)
```

In [509…
```
# df['change_of_state'] = int(df['position_state'].str == df['last_position_state'].str)
```

In [510…
```
# df['change_of_state'].mean()
```

In [513…
```
pd.set_option('display.max_colwidth', None)
df[['t', 'position_state', 'x_pos', 'y_pos']].iloc[50:55]
```

Out[513…

	t	position_state	x_pos	y_pos
50	2.55	XAPPPOFNRRGIJIWCLCMNXAUULYJJZMWAACGPTMRQZGWAEXDRPWQNATTIOKFEDDJR	0.820531	0.802477
51	2.60	XAPPPOFNRRGIJIWCLCMNXAUULYJJZMWAACGPTMRQZGWAEXDRPWQNATTIOKFEDDJR	0.81486	0.80559
52	2.65	XAPPPOFNLCMNXAUUACGPTMRQRRGIJIWCLYJJZMWAZGWAEXDRPWQNATTIOKFEDDJR	0.808613	0.807819
53	2.70	XAPPPOFNLCMNXAUUACGPTMRQLYJJZMWARRGIJIWCZGWAEXDRPWQNATTIOKFEDDJR	0.800055	0.808812
54	2.75	ACGPTMRQLCMNXAUUXAPPPOFNLYJJZMWARRGIJIWCZGWAEXDRPWQNATTIOKFEDDJR	0.794426	0.809313

In [ ]: